Esercizio

- **esercizio 1 del 30/03/2015**

(Svolto a lezione, vedere soluzioni sul sito dei compiti)

File disordinato, costi

- Sia per ricerca, sia per inserimento:
  - costo di solito lineare (nel numero di blocchi del file)
- Osservazioni (marginali)
  - se il record è presente, il costo sarà mediamente pari alla metà del numero di blocchi, ma possiamo trascurare
  - per l'inserimento, potremmo mantenere un riferimento all'ultimo blocco (però in caso di vincoli da verificare, ad esempio chiave, la scansione ci servirebbe comunque)

Struttura ordinata

- Ogni record (ennupla) ha una posizione basata sul valore di un campo (detto **chiave** o, meglio, **pseudochiave**)

Ordinato per Matricola

Matricola	Cognome	Nome
15	Neri	Piero
21	Rossi	Mario
30	Bianchi	Gino
38	Verdi	Luigi
40	Rossi	Mario
53	Neri	Luca

Ordinato per Cognome

Matricola	Cognome	Nome
30	Bianchi	Gino
15	Neri	Piero
53	Neri	Luca
21	Rossi	Mario
40	Rossi	Mario
38	Verdi	Luigi

Struttura ordinata (2)

- Gli aggiornamenti dovrebbero essere coerenti
  - possono richiedere modifiche significative
    - inserimenti
    - campi a lunghezza variabile con valori più lunghi dei precedenti

Matricola	Cognome	Nome
30	Bianchi	Gino
15	Neri	Piero
53	Neri	Luca
21	Rossi	Mario
40	Rossi	Mario
38	Verdi	Luigi

66	Bruni	Marco

Struttura ordinata (2)

- Gli aggiornamenti dovrebbero essere coerenti
  - possono richiedere modifiche significative
    - inserimenti
    - campi a lunghezza variabile con valori più lunghi dei precedenti

Matricola	Cognome	Nome
30	Bianchi	Gino
15	Neri	Piero
53	Neri	Luca
21	Rossi	Mario
40	Rossi	Mario
38	Verdi	Luigi

66	Bruni	Marco

Matricola	Cognome	Nome
30	Bianchi	Gino
15	Neri	Piero
53	Neri	Luca
21	Rossi	Mario
40	Rossi	Mario
38	Verdi	Luigi

Struttura ordinata (2)

- Gli aggiornamenti dovrebbero essere coerenti
  - possono richiedere modifiche significative
    - inserimenti
    - campi a lunghezza variabile con valori più lunghi dei precedenti

Matricola	Cognome	Nome
30	Bianchi	Gino
15	Neri	Piero
53	Neri	Luca
21	Rossi	Mario
40	Rossi	Mario
38	Verdi	Luigi

66	Bruni	Marco

Matricola	Cognome	Nome
30	Bianchi	Gino

15	Neri	Piero
53	Neri	Luca
21	Rossi	Mario
40	Rossi	Mario
38	Verdi	Luigi

Struttura ordinata (2)

- Gli aggiornamenti dovrebbero essere coerenti
  - possono richiedere modifiche significative
    - inserimenti
    - campi a lunghezza variabile con valori più lunghi dei precedenti

Matricola	Cognome	Nome
30	Bianchi	Gino
15	Neri	Piero
53	Neri	Luca
21	Rossi	Mario
40	Rossi	Mario
38	Verdi	Luigi

66	Bruni	Marco

Matricola	Cognome	Nome
30	Bianchi	Gino
66	Bruni	Marco
15	Neri	Piero
53	Neri	Luca
21	Rossi	Mario
40	Rossi	Mario
38	Verdi	Luigi

Struttura ordinata (3)

- Soluzioni (anche combinate)
  - riorganizzazione immediata
  - spazio inizialmente ridondante
  - inserimenti in coda o in blocchi di overflow
  - riorganizzazioni periodiche

- I DBMS di solito non mantengono l'ordinamento:
  - si ordina una tantum e poi si riordina quando serve

Strutture ordinate

- Permettono ricerche binarie ma …
  - … solo in alcuni casi
    - come troviamo la "metà" del file?
    - e poi la metà della metà? …
- Nelle basi di dati relazionali si utilizzano quasi solo in combinazione con indici (file ISAM o file ordinati con indice primario)
- Comunque, sono utili
  - per fornire risultati ordinati
  - in preparazione per altre operazioni

Strutture ordinate, costo

- Senza ulteriori strutture, non è facile valutare il costo della ricerca (ma di solito le altre strutture ci sono)
- Per l'inserimento, come nelle strutture disordinate (o peggio, se si vuole mantenere l'ordinamento)
- Per la scansione ordinata (secondo il campo di ordinamento):
  - costo lineare (mentre per I file disordinati è necessario ordinare, vedremo più avanti il costo)

File hash

- Accesso diretto o associativo (cioè accesso sulla base del valore di un campo) molto efficiente
  - In SimpleDB, invece del metodo beforeFirst() che abbiamo nel file disordinato (o ordinato), abbiamo un metodo
    - beforeFirst(searchKey)
- La tecnica si basa su quella utilizzata per le tavole hash in memoria centrale

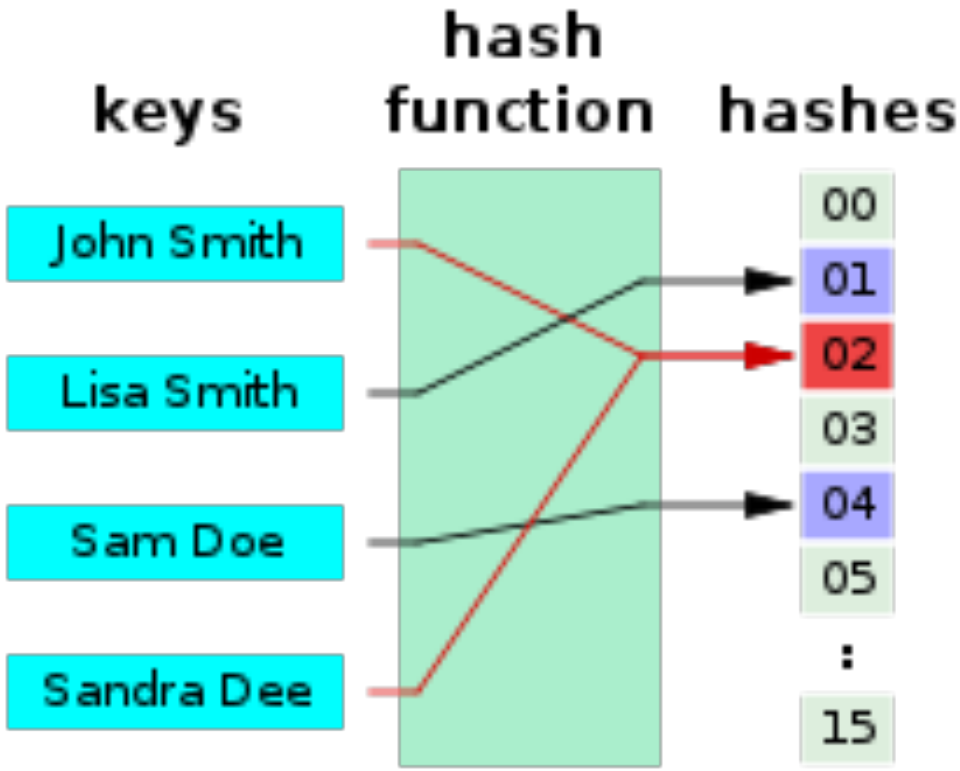
Tavola hash

- Obiettivo:
  - accesso diretto ad un insieme di record sulla base del valore di un campo (detto **chiave**, che per semplicità supponiamo identificante, ma non è necessario)
- Se i possibili valori della chiave sono in numero paragonabile al numero di record (e corrispondono ad un "tipo indice") allora usiamo un array; ad esempio: università con 1000 studenti e numeri di matricola compresi fra 1 e 1000 o poco più e file con tutti gli studenti
- Se i possibili valori della chiave sono molti di più di quelli effettivamente utilizzati, non possiamo usare l'array (spreco);
- ad esempio: 40 studenti e matricola di 6 cifre (un milione di possibili chiavi)

Tavola hash, 2

- Volendo continuare ad usare qualcosa di simile ad un array, ma senza sprecare spazio, possiamo pensare di trasformare i valori della chiave in possibili indici di un array:
  - **funzione hash**:
    - associa ad ogni valore della chiave un "indirizzo", in uno spazio di dimensione paragonabile (leggermente superiore) a quello strettamente necessario
    - poiché il numero di possibili chiavi è molto maggiore del numero di possibili indirizzi ("lo spazio delle chiavi è più grande dello spazio degli indirizzi"), la funzione non può essere iniettiva e quindi esiste la possibilità di collisioni (chiavi diverse che corrispondono allo stesso indirizzo)
    - le buone funzioni hash distribuiscono in modo casuale e uniforme, riducendo la probabilità di collisione (che si riduce aumentando lo spazio ridondante)
    - esempio (utile didatticamente, ma non efficace in generale): funzione mod (resto della divisione fra interi)

Tavola hash, 3



Fonte: Wikipedia

Un esempio semplicissimo (alla lavagna)

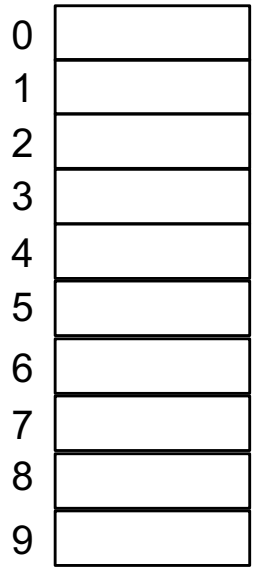
- 8 record con chiavi

  240772
  240810
  449726
  447004
  453900
  281425
  281267
  405154

- tavola hash con 10 posizioni
- funzione hash
  - K mod 10

Un esempio semplicissimo (alla lavagna)

- 8 record con chiavi
  - 240772
  - 240810
  - 449726
  - 447004
  - 453900
  - 281425
  - 281267
  - 405154
- tavola hash con 10 posizioni
- funzione hash
  - K mod 10



Un esempio semplicissimo (alla lavagna)

- 8 record con chiavi
  - 240772 ✓
  - 240810 ✓
  - 449726 ✓
  - 447004 ✓
  - 453900 ✗
  - 281425 ✓
  - 281267 ✓
  - 405154 ✗
- tavola hash con 10 posizioni
- funzione hash
  - K mod 10

0	240810
1	
2	240772
3	
4	447004
5	281425
6	449726
7	281267
8	
9	

453900
405154

Tavola hash, gestione delle collisioni

- Varie tecniche:
  - posizioni successive disponibili
  - tabella di overflow (gestita in forma collegata)
  - funzioni hash "alternative"

Tavola hash, un esempio semplice

- 40 record
- tavola hash con 50 posizioni:

0	60600
1	66301
2	205802
4	200604
5	66005
9	201159
10	205610
12	205912
14	200464
17	205617
18	200268
19	205619

22	210522
24	205724
27	205977
28	205478
30	200430
33	210533
37	205887
38	102338

40	102690
41	115541
42	206092
43	205693
45	205845
46	205796
48	200498
49	206049

1	205751
2	200902
2	116202
5	116455
5	200205
10	201260
10	102360
10	205460
12	205762
17	205667
38	200138
46	200296

Tavola hash, quante collisioni?

- le collisioni ci sono (quasi) sempre
  - anche molte
- Ma
  - le collisioni multiple hanno probabilità che decresce al crescere della molteplicità
  - la molteplicità media delle collisioni è molto bassa

Le chiavi e le collisioni

- 40 record
- tavola hash con 50 posizioni:
  - 1 collisione a 4
  - 2 collisioni a 3
  - 5 collisioni a 2
  - 20 record senza collisione

M	M mod 50	M	M mod 50
60600	0	200268	18
66301	1	205619	19
205751	1	210522	22
205802	2	205724	24
200902	2	205977	27
116202	2	205478	28
200604	4	200430	30
66005	5	210533	33
116455	5	205887	37
200205	5	200138	38
201159	9	102338	38
205610	10	102690	40
201260	10	115541	41
102360	10	206092	42
205460	10	205693	43
205912	12	205845	45
205762	12	200296	46
200464	14	205796	46
205617	17	200498	48
205667	17	206049	49

L'esempio semplice, "costo"

- 40 record
- tavola hash con 50 posizioni:
  - 1 collisione a 4
  - 2 collisioni a 3
  - 5 collisioni a 2
  - 20 record senza collisione
- numero medio di accessi:
  (28x1+8x2+3x3+1x4)/40 = 1,425

M	M mod 50	M	M mod 50
60600	0	200268	18
66301	1	205619	19
205751	1	210522	22
205802	2	205724	24
200902	2	205977	27
116202	2	205478	28
200604	4	200430	30
66005	5	210533	33
116455	5	205887	37
200205	5	200138	38
201159	9	102338	38
205610	10	102690	40
201260	10	115541	41
102360	10	206092	42
205460	10	205693	43
205912	12	205845	45
205762	12	200296	46
200464	14	205796	46
205617	17	200498	48
205667	17	206049	49

Funzione di hash, proprietà (1)

- le collisioni ci sono (quasi) sempre
  - anche molte
- Ma
  - le collisioni multiple hanno probabilità che decresce al crescere della molteplicità
  - la molteplicità media delle collisioni è molto bassa

File hash

- L'idea è la stessa della tavola hash, ma si basa sull'organizzazione in blocchi:
  - ogni blocco contiene più record
  - lo spazio degli indirizzi è più piccolo
    - nell’esempio, con fattore di blocco pari a 10, possiamo usare “mod 5” invece di “mod 50”
- Le collisioni (overflow) sono di solito gestite con blocchi collegati

Paragoni stimolanti

- Perché nel calcio spesso
  - il primo portiere di riserva gioca pochissimo?
  - mentre il primo difensore di riserva gioca molto spesso?
- Perché nel rugby, su otto giocatori in panchina,
  - cinque o sei sono "avanti" (cioè giocatori di mischia, in cui ci sono otto ruoli)
  - solo due o tre sono di altri ruoli (che sono sette)?

File hash

- L'idea è la stessa della tavola hash, ma si basa sull'organizzazione in blocchi:
  - ogni blocco contiene più record
  - lo spazio degli indirizzi è più piccolo
    - nell’esempio, con fattore di blocco pari a 10, possiamo usare “mod 5” invece di “mod 50”
- Le collisioni (overflow) sono di solito gestite con blocchi collegati

L'esempio semplicissimo sulla tavola hash

- 8 record con chiavi
  - 240772 ✓
  - 240810 ✓
  - 449726 ✓
  - 447004 ✓
  - 453900 ✗
  - 281425 ✓
  - 281267 ✓
  - 405154 ✗
- tavola hash con 10 posizioni
- funzione hash
  - K mod 10

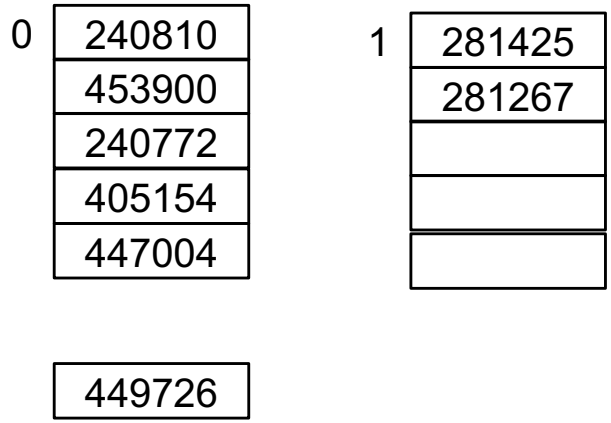
0	240810
1	
2	240772
3	
4	447004
5	281425
6	449726
7	281267
8	
9	

453900
405154

L'esempio semplicissimo

– 8 record, fattore di blocco 5, funzione mod 2, due blocchi

240772
240810
449726
447004
453900
281425
281267
405154



File hash per l'esempio semplice

0	1	2	3	4
60600	66301	205802	200268	200604
66005	205751	200902	205478	201159
116455	115541	116202	210533	200464
200205	200296	205912	200138	205619
205610	205796	205762	102338	205724
201260		205617	205693	206049
102360		205667	200498	
205460		210522		
200430		205977		
102690		205887		
205845		206092		

Nell’esempio semplice

- 40 record
- **tavola hash con 50 posizioni**
  - 1 collisione a 4
  - 2 collisioni a 3
  - 5 collisioni a 2
  - in totale, 12 record in overflow

  numero medio di accessi: 1,425
- **file hash con fattore di blocco 10**;
  5 blocchi con 10 posizioni ciascuno:
  - due soli record in overflow

  numero medio di accessi: (42/40) = 1,05
- perché?

Collisioni, stima

- Lunghezza media delle catene di overflow, al variare di
  - Numero di record esistenti: T
  - Numero di blocchi: B
  - Fattore di blocco: F
  - Coefficiente di riempimento: T/(FxB)

	1	2	3	5	10	(F)
.5	0.5	0.177	0.087	0.031	0.005	
.6	0.75	0.293	0.158	0.066	0.015	
.7	1.167	0.494	0.286	0.136	0.042	
.8	2.0	0.903	0.554	0.289	0.110	
.9	4.495	2.146	1.377	0.777	0.345	
T/(FxB)						

Costo, stima

- Costo (uno più la lunghezza media), nelle stesse condizioni
  - Numero di record esistenti: T
  - Numero di blocchi: B
  - Fattore di blocco: F
  - Coefficiente di riempimento: T/(FxB)

	1	2	3	5	10	(F)
.5	1.5	1.177	1.087	1.031	1.005	
.6	1.75	1.293	1.158	1.066	1.015	
.7	2.167	1.494	1.286	1.136	1.042	
.8	3.0	1.903	1.554	1.289	1.110	
.9	5.495	3.146	2.377	1.777	1.345	
T/(FxB)						

File hash, osservazioni

- Costo medio di poco superiore all'unità
    - il caso peggiore è molto costoso ma talmente improbabile da poter essere ignorato
  - è l'organizzazione più efficiente per l'accesso diretto basato su valori della chiave con condizioni di uguaglianza (accesso puntuale)
- Non è efficiente per ricerche basate su intervalli
- I file hash "degenerano" se si riduce lo spazio sovrabbondante: funzionano solo con file la cui dimensione non varia molto nel tempo
  - Esistono però tecniche per superare questo difetto:
    - Hashing dinamico (estendibile, lineare)

Hashing estendibile

- Il numero di blocchi (o bucket) non è predefinito
- Utilizza una directory dei blocchi (anch'essa di dimensione variabile, anche se molto lentamente)

- Omettiamo i dettagli

Hashing estensibile, commenti

- Costo aggiuntivo, l'accesso alla directory
  - Costo due, directory più blocco (più le riorganizzazioni, molto rare, quindi possiamo assumere due come riferimento, la media sarà di poco superiore a due)
  - Però se si usa di frequente, la directory può essere nei buffer e quindi il costo si avvicina all'unità
- Supera le limitazioni dell'hashing statico in caso di dimensione variabile nel tempo

Strutture primarie e secondarie

- Ricordiamo
  - strutture **primarie**: determinano collocazione/posizionamento dei record
  - strutture **secondarie**: elementi aggiuntivi per l'accesso efficiente
- Le strutture sequenziali (disordinate, ordinate, etc.) sono strutture **primarie**
- Le strutture hash, come viste finora, sono pure **primarie** (se ne può avere una variante secondaria)
- Vediamo ora strutture che nascono come **secondarie** (anche se loro varianti possono essere primarie)
- Cominciamo con alcuni esempi

Una biblioteca e il suo catalogo

Indice analitico

Abort, 190
Access, *419–440*
    Avvio di, 420
    Cammini di join in, 426
    Definizione di tabelle in, 421
    Domini in, 421–425
    Interprete SQL in, 435
    Macro in, 438
    Maschere in, 436
    Popolamento delle tabelle in, 428
    QBE (query by example) in, 429–435
    Report in, 438
Accesso, Controllo di, 187
Accorpamento
    di associazioni, 320, 330–331
    di entità, 320, 330
Acidità delle transazioni, 191
ADO, ActiveX Data Object, 213
ANSI, 111
any, 155
Applicazioni
    in DB2, 451–455
asc, 142
Asserzioni in SQL, 176
Associazione
    verifica di normalizzazione, 409–410
Associazioni
    Accorpamento di, 320, 330–331
    in UML, 256, 258–260
    nel modello E-R, 236–238
    Partizionamento di, 320, 330–331
Atomicità, 191
Attributi
    Cardinalità di, 243–244
    composti, 240
    di relazione, 37
    Eliminazione di — multivalore, 329
    in UML, 257
    nel modello E-R, 238–240
Autorizzazioni, 19, 187
avg, 144

Indice

Prefazione ix

1 Introduzione 15
1.1 Sistemi informativi, informazioni e dati 15
1.2 Basi di dati e sistemi di gestione di basi di dati 17
1.3 Modelli dei dati 20
1.4 Linguaggi e utenti delle basi di dati 24
1.5 Vantaggi e svantaggi dei DBMS 27

Parte Prima Basi di dati relazionali: modello e linguaggi 31

2 Il modello relazionale 33
2.1 Il modello relazionale: strutture 33
2.2 Vincoli di integrità 47
2.3 Conclusioni 57

Indici di file

- **Indice**:
  - struttura ausiliaria per l'accesso (efficiente) ai record di un file sulla base dei valori di un campo (o di una "concatenazione di campi") detto **chiave** (o, meglio, **pseudochiave**, perché non è necessariamente identificante);
- Idea fondamentale: l'indice analitico di un libro: lista di coppie (termine,pagina), ordinata alfabeticamente sui termini, posta in fondo al libro e separabile da esso
- Un indice ***I*** di un file ***F*** è un altro file, con record a due campi: chiave e indirizzo (dei record di ***F*** o dei relativi blocchi), ordinato secondo i valori della chiave

Tipi di indice

- indice **primario**:
  - su un campo sul cui ordinamento è basata la memorizzazione (detti anche indici di **cluster**, anche se talvolta si chiamano **primari** quelli su una chiave identificante e di **cluster** quelli su una pseudochiave non identificante)
  - Attenzione:
    - In alcuni sistemi viene chiamato "indice primario" un indice (di solito secondario) definito sulla chiave primaria. Noi **non** utilizziamo questa terminologia
- indice **secondario**
  - su un campo con ordinamento diverso da quello di memorizzazione

Indice analitico

Abort, 190
Access, *419–440*
    Avvio di, 420
    Cammini di join in, 426
    Definizione di tabelle in, 421
    Domini in, 421–425
    Interprete SQL in, 435
    Macro in, 438
    Maschere in, 436
    Popolamento delle tabelle in, 428
    QBE (query by example) in, 429–435
    Report in, 438
Accesso, Controllo di, 187
Accorpamento
    di associazioni, 320, 330–331
    di entità, 320, 330
Acidità delle transazioni, 191
ADO, ActiveX Data Object, 213
ANSI, 111
any, 155
Applicazioni
    in DB2, 451–455
asc, 142
Asserzioni in SQL, 176
Associazione
    verifica di normalizzazione, 409–410
Associazioni
    Accorpamento di, 320, 330–331
    in UML, 256, 258–260
    nel modello E-R, 236–238
    Partizionamento di, 320, 330–331
Atomicità, 191
Attributi
    Cardinalità di, 243–244
    composti, 240
    di relazione, 37
    Eliminazione di — multivalore, 329
    in UML, 257
    nel modello E-R, 238–240
Autorizzazioni, 19, 187
avg, 144

Indice

Prefazione **ix**

1 Introduzione **15**
1.1 Sistemi informativi, informazioni e dati 15
1.2 Basi di dati e sistemi di gestione di basi di dati 17
1.3 Modelli dei dati 20
1.4 Linguaggi e utenti delle basi di dati 24
1.5 Vantaggi e svantaggi dei DBMS 27

Parte Prima Basi di dati relazionali: modello e linguaggi **31**

2 Il modello relazionale **33**
2.1 Il modello relazionale: strutture 33
2.2 Vincoli di integrità 47
2.3 Conclusioni 57

Tipi di indice, commenti

- Nell'esempio del libro
  - indice generale: primario (pagine in ordine)
  - indice analitico: secondario (pagine in disordine)
- Ogni file può avere al più un indice primario (salvo il caso di un indice su un "prefisso" dell'altro, ad esempio uno su cognome e l'altro su cognome e nome) e un numero qualunque di indici secondari (su campi diversi). Esempio:
  - In alcune guide turistiche abbiamo due indici "secondari"
    - indice dei luoghi
    - indice degli artisti
  - Le biblioteche hanno (o avevano) cataloghi
    - per titolo
    - per autore
    - per argomento
- Un file hash non può avere un indice primario